

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
«МАИ»

Лабораторная работа №1
по дисциплине «Программирование»
на тему:
«Двумерные массивы»
Вариант №8

Выполнили:
студенты гр. М4О-208Б-23
Порошин Г.А.
Прошин Е.С.
Проверили:
Чечиков Ю.Б.
Секретарев В.Е.

Г.Москва
2025

Содержание

Постановка задачи	2
Блок-схема	3
Листинг рабочей программы	6
Тесты	10
Некорректные тесты	10
Корректные тесты	11
Вывод	16

Постановка задачи

Кафедра 304

Курс: ИНФОРМАТИКА II семестр

Задание 4: *Двумерные массивы*

ВАРИАНТ № 8

Дана целочисленная квадратная матрица. Определить:

1. Произведение положительных элементов матрицы, которые расположены над побочной диагональю.

2. Максимум среди сумм по строкам нечетных элементов матрицы.

Используя универсальные для различных наборов исходных данных подпрограммы реализовать данный алгоритм для заданных матриц: $A(N,N)$, $B(M,M)$.

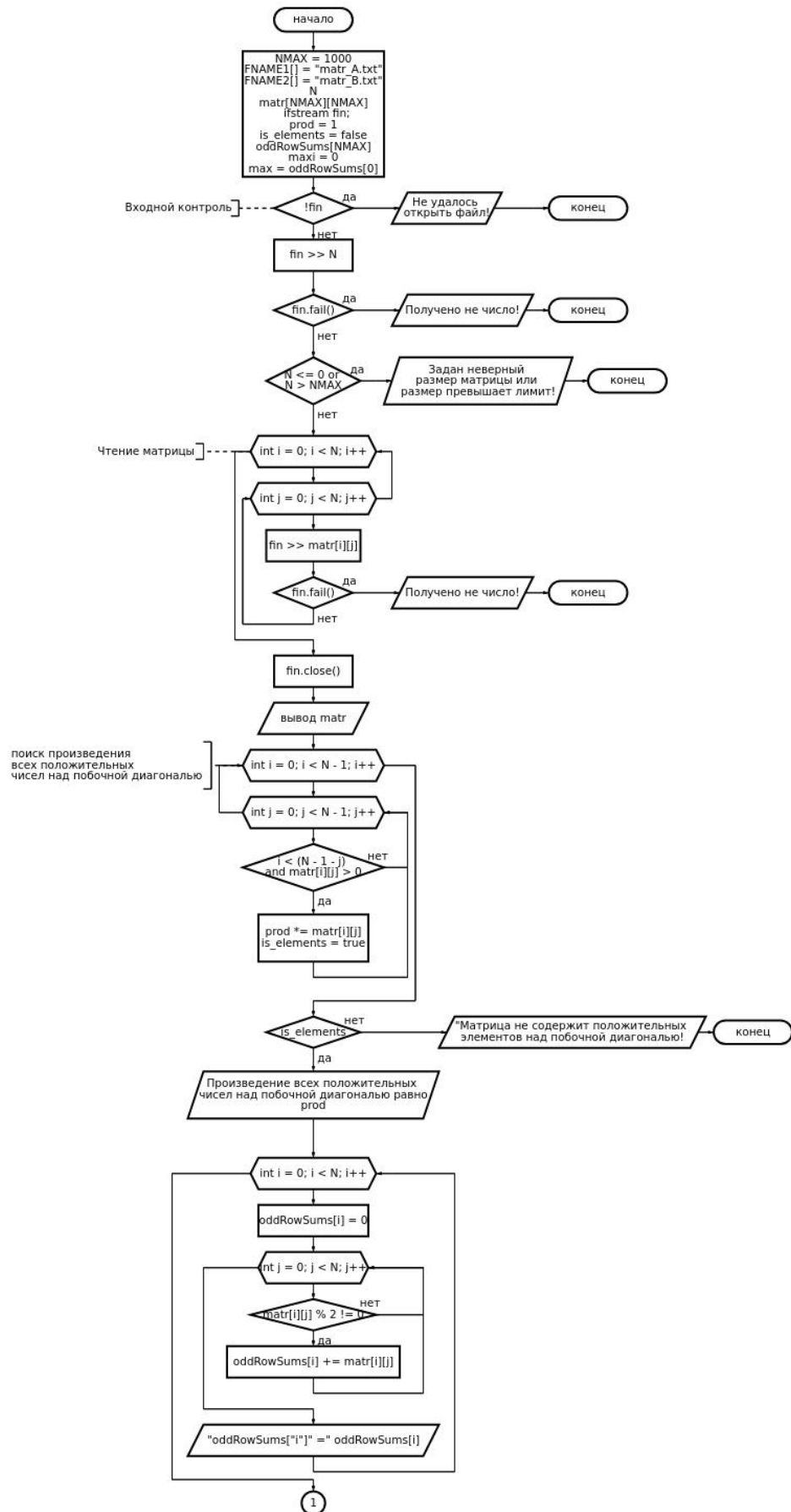
В качестве одного из вариантов исходных данных принять: $N=7$, $M=9$.

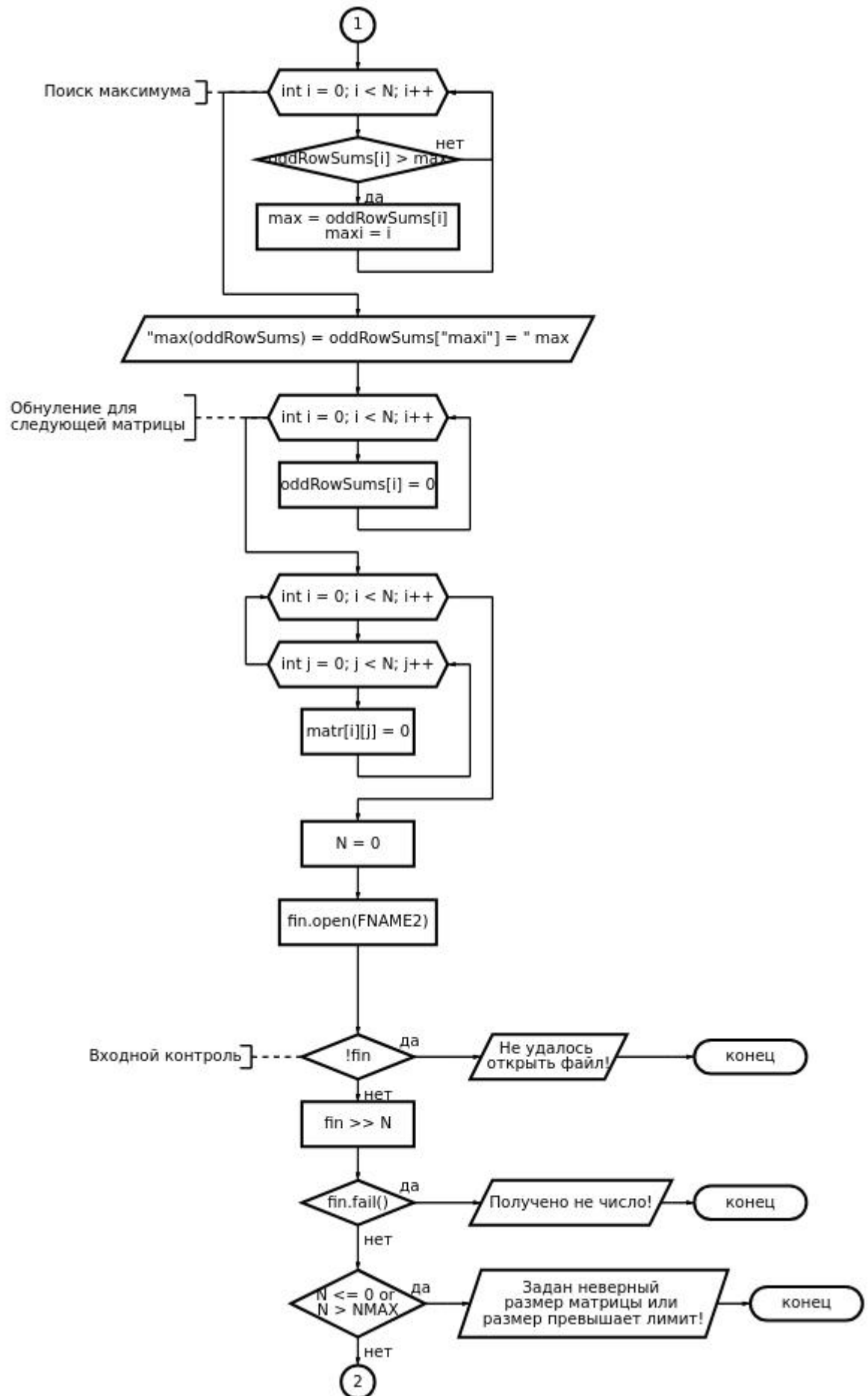
Чтение данных их файла производить с использованием функций ввода/вывода языка C++.

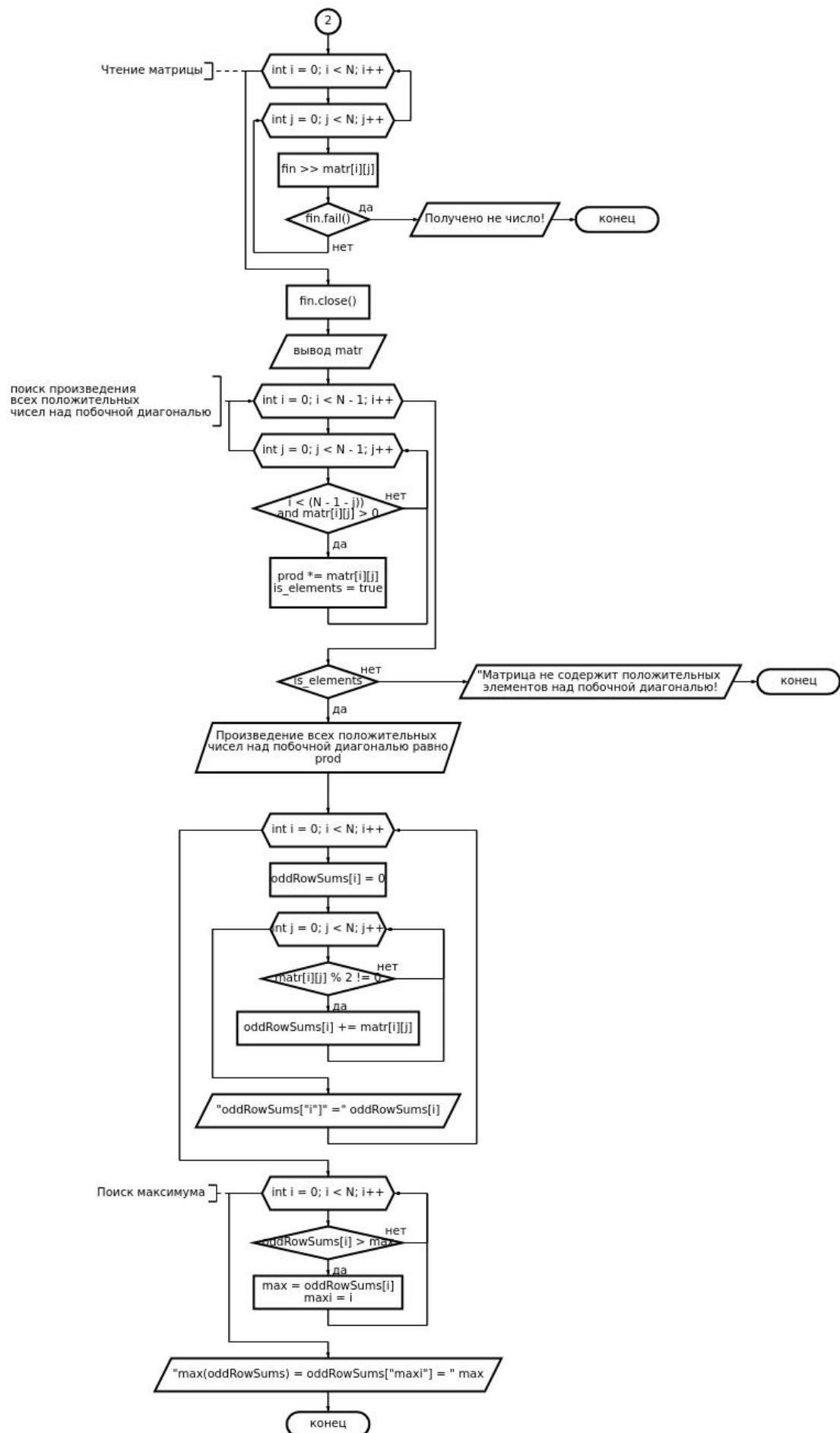
Алгоритм должен быть параметризован; обмен данными с подпрограммой должен осуществляться только через параметры; каждый из наборов исходных данных хранится в отдельном файле.

Реализовать программу в двух вариантах: в первом – при обращении к элементам массива использовать индексы, во втором – работать с динамическим массивом через указатели.

Блок-схема







Листинг рабочей программы

```
// Creators: Порошин Г. А. , Прошин Е. С.
#include <fstream>
#include <iostream>

using namespace std;

const int NMAX = 1000;
const char FNAME1[] = "matr_A.txt";
const char FNAME2[] = "matr_B.txt";

int main() {
    // инициализация переменных
    int N;
    int matr[NMAX][NMAX];
    ifstream fin;
    fin.open(FNAME1);
    // Матрица A
    // входной контроль
    if (!fin) {
        cout << "Не удалось открыть файл!\n";
        return 1;
    }
    fin >> N;
    if (fin.fail()) {
        cout << "Получено не число!\n";
        return 4;
    }
    // считывает размера и матрицы из файла
    if (N <= 0 or N > NMAX) {
        cout << "Задан неверный размер матрицы или размер превышает лимит!\n";
        return 3;
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fin >> matr[i][j];
            if (fin.fail()) {
                cout << "Получено не число!\n";
                return 4;
            }
        }
    }
    fin.close(); // закрытие файла
    // вывод матрицы
    cout << "\nA[" << N << "x" << N << "]=\n";
    for (int i = 0; i < N; i++) {
        cout << "\t";
        for (int j = 0; j < N; j++) {
            cout << matr[i][j] << "\t";
        }
        // печать скобок
        if (i == 0)
            cout << "\r\r";
        else if (i == N - 1)
            cout << "\r\r";
        else
            cout << "\r|";
        cout << "\n";
    }
}
```

```

// первое задание: поиск произведения всех
// положительных чисел над побочной диагональю
cout << "\n";

// инициализация переменных
int prod = 1;
bool is_elements = false;

// поиск элементов и подсчёт произведения
for (int i = 0; i < N - 1; i++) {
    for (int j = 0; j < N - 1; j++) {
        if ((i < (N - 1 - j)) and matr[i][j] > 0) {
            prod *= matr[i][j];
            is_elements = true;
        }
    }
}

// проверка на положительные элементы
if (is_elements) {
    cout << "Задание 1.\n";
    cout << "\tПроизведение всех положительных\n\tчисел над побочной "
        << "диагональю равно "
        << prod << endl;
} else {
    cout << "Матрица не содержит положительных элементов над побочной "
        << "диагональю!\n";
}

// второе задание: поиск максимума среди сумм
// по строкам нечётных элементов матрицы
cout << "\nЗадание 2.\n";
cout << "\tСуммы по строкам нечётных элементов матрицы равны:\n";

// инициализация переменных
int oddRowSums[NMAX];

// посчёт и вывод сумм
for (int i = 0; i < N; i++) {
    oddRowSums[i] = 0;
    for (int j = 0; j < N; j++) {
        if (matr[i][j] % 2 != 0)
            oddRowSums[i] += matr[i][j];
    }
    cout << "\t\toddRowSums[" << i << "]= " << oddRowSums[i] << "\n";
}

// инициализация переменных
int maxi = 0;
int max = oddRowSums[0];
// поиск максимума
for (int i = 0; i < N; i++) {
    if (oddRowSums[i] > max) {
        max = oddRowSums[i];
        maxi = i;
    }
}
cout << "\n\tmax(oddRowSums)=oddRowSums[" << maxi << "]= " << max << "\n";

// обнуление oddRowSums
for (int i = 0; i < N; i++)

```

```

    oddRowSums[i] = 0;
// очистка матрицы
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        matr[i][j] = 0;
N = 0;
// Матрица B
fin.open(FNAME2);
// входной контроль
if (!fin) {
    cout << "Не удалось открыть файл!\n";
    return 1;
}
fin >> N;
if (fin.fail()) {
    cout << "получено не число!\n";
    return 4;
}
// считывает размера и матрицы из файла
if (N <= 0 or N > NMAX) {
    cout << "Задан неверный размер матрицы или размер превышает лимит!\n";
    return 3;
}
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        fin >> matr[i][j];
        if (fin.fail()) {
            cout << "получено не число!\n";
            return 4;
        }
    }
}
fin.close(); // закрытие файла
// вывод матрицы
cout << "\nB[" << N << "x" << N << "]=\n";
for (int i = 0; i < N; i++) {
    cout << "\t";
    for (int j = 0; j < N; j++) {
        cout << matr[i][j] << "\t";
    }
    // печать скобок
    if (i == 0)
        cout << "\r";
    else if (i == N - 1)
        cout << "\r^";
    else
        cout << "\r|";
    cout << "\n";
}
// первое задание: поиск произведения всех
// положительных чисел над побочной диагональю
cout << "\n";
// инициализация переменных
prod = 1;
is_elements = false;
// поиск элементов и подсчёт произведения
for (int i = 0; i < N - 1; i++) {
    for (int j = 0; j < N - 1; j++) {
        if ((i < (N - 1 - j)) and matr[i][j] > 0) {
            prod *= matr[i][j];
            is_elements = true;
        }
    }
}

```

```

    }
}
// проверка на положительные элементы
if (is_elements) {
    cout << "Задание 1.\n";
    cout << "\tПроизведение всех положительных\n\tчисел над побочной "
           "диагональю равно "
           << prod << endl;
} else {
    cout << "Матрица не содержит положительных элементов над побочной "
           "диагональю!\n";
}
// второе задание: поиск максимума среди сумм
// по строкам нечётных элементов матрицы
cout << "\nЗадание 2.\n";
cout << "\tСуммы по строкам нечётных элементов матрицы равны:\n";
// посчёт и вывод сумм
for (int i = 0; i < N; i++) {
    oddRowSums[i] = 0;
    for (int j = 0; j < N; j++) {
        if (matr[i][j] % 2 != 0)
            oddRowSums[i] += matr[i][j];
    }
    cout << "\t\toddRowSums[" << i << "]=" << oddRowSums[i] << "\n";
}
// инициализация переменных
maxi = 0;
max = oddRowSums[0];
// поиск максимума
for (int i = 0; i < N; i++) {
    if (oddRowSums[i] > max) {
        max = oddRowSums[i];
        maxi = i;
    }
}
cout << "\n\tmax(oddRowSums)=oddRowSums[" << maxi << "]=" << max << "\n";
// обнуление oddRowSums
for (int i = 0; i < N; i++)
    oddRowSums[i] = 0;
// очистка матрицы
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        matr[i][j] = 0;
N = 0;
return 0;
}

```

Тесты

Некорректные тесты

1.

Цель: проверить работу программы при отсутствии файла

Исходные данные: -

Ожидаемый результат: Не удалось открыть файл!

Результат:

Не удалось открыть файл!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

2.

Цель: проверить работу программы при неверном чтении

Исходные данные: -

Ожидаемый результат: Получено не число!

Результат:

Получено не число!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

3.

Цель: проверить работу программы при неправильном размере матрицы

Исходные данные: -2 1 1 1 1

Ожидаемый результат: Задан неверный размер матрицы!

Результат:

Задан неверный размер матрицы!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

4.

Цель: проверить работу программы при отсутствии положительных чисел над побочной диагональю

Исходные данные: 2 -1 1 1 1

Ожидаемый результат: Матрица не содержит положительных элементов над побочной диагональю!

Результат:

Матрица не содержит положительных элементов над побочной диагональю!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

Корректные тесты

1.

Цель: проверить работу программы при корректных данных

Исходные данные:

7

-2	-3	0	7	-7	-9	-1
-5	3	7	10	4	-9	-2
1	-8	-6	6	-6	-5	0
10	-5	1	-5	6	1	10
-3	-7	-1	-6	1	-6	-10
-2	8	4	-3	4	7	-2
-9	-2	-8	6	-5	-5	-8

9

-7	10	-8	-8	1	-3	-10	-6	-7
0	-8	7	-6	-5	9	-10	1	7
0	-10	-1	2	7	10	10	-5	-10
-3	3	5	-6	-5	2	-1	7	3
-2	9	-4	-3	2	-2	7	8	4
4	-4	-2	1	1	-7	-2	-3	10
3	-7	-1	5	2	-6	-1	7	9
8	-4	0	-9	-3	-5	7	1	4
-8	3	8	-3	-5	-2	-6	10	-6

Ожидаемый результат:

A[7x7]=

-2	-3	0	7	-7	-9	-1
-5	3	7	10	4	-9	-2
1	-8	-6	6	-6	-5	0
10	-5	1	-5	6	1	10
-3	-7	-1	-6	1	-6	-10
-2	8	4	-3	4	7	-2
-9	-2	-8	6	-5	-5	-8

Задание 1.

Произведение всех положительных
чисел над побочной диагональю равно 352800

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=-13
oddRowSums[1]=-4
oddRowSums[2]=-4
oddRowSums[3]=-8
oddRowSums[4]=-10
oddRowSums[5]=4
oddRowSums[6]=-19
```

```
max(oddRowSums)=oddRowSums[5]=4
```

B[9x9]=

-7	10	-8	-8	1	-3	-10	-6	-7
0	-8	7	-6	-5	9	-10	1	7

0	-10	-1	2	7	10	10	-5	-10
-3	3	5	-6	-5	2	-1	7	3
-2	9	-4	-3	2	-2	7	8	4
4	-4	-2	1	1	-7	-2	-3	10
3	-7	-1	5	2	-6	-1	7	9
8	-4	0	-9	-3	-5	7	1	4
-8	3	8	-3	-5	-2	-6	10	-6

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 1143072000

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=-16
oddRowSums[1]=19
oddRowSums[2]=1
oddRowSums[3]=9
oddRowSums[4]=13
oddRowSums[5]=-8
oddRowSums[6]=15
oddRowSums[7]=-9
oddRowSums[8]=-5
```

```
max(oddRowSums)=oddRowSums[1]=19
```

Результат:

A[7x7]=

-2	-3	0	7	-7	-9	-1
-5	3	7	10	4	-9	-2
1	-8	-6	6	-6	-5	0
10	-5	1	-5	6	1	10
-3	-7	-1	-6	1	-6	-10
-2	8	4	-3	4	7	-2
-9	-2	-8	6	-5	-5	-8

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 352800

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=-13
oddRowSums[1]=-4
oddRowSums[2]=-4
oddRowSums[3]=-8
oddRowSums[4]=-10
oddRowSums[5]=4
oddRowSums[6]=-19
```

```
max(oddRowSums)=oddRowSums[5]=4
```

```
B[9x9]=
[
  -7    10   -8   -8    1   -3   -10   -6   -7
   0    -8    7   -6   -5    9   -10    1    7
   0   -10   -1    2    7   10   10   -5  -10
  -3    3    5   -6   -5    2   -1    7    3
  -2    9   -4   -3    2   -2    7    8    4
   4   -4   -2    1    1   -7   -2   -3   10
   3   -7   -1    5    2   -6   -1    7    9
   8   -4    0   -9   -3   -5    7    1    4
  -8    3    8   -3   -5   -2   -6   10   -6
]
```

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 1143072000

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=-16
oddRowSums[1]=19
oddRowSums[2]=1
oddRowSums[3]=9
oddRowSums[4]=13
oddRowSums[5]=-8
oddRowSums[6]=15
oddRowSums[7]=-9
oddRowSums[8]=-5
```

```
max(oddRowSums)=oddRowSums[1]=19
```

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

2.

Цель: проверить работу программы при большем количестве пробелов в строке

Исходные данные:

```
7
8    -8    4    4    1   -4   -1
-6    6    3  -10   -3   -8    9
-8   -3    8   -9    1    1    3
-4    6    8    2   -9   -2   -9
9   -9   -2   10   10    5    3
-1  -10   10   -9  -10   -6    6
9   -6   -7   -3    2    4   -5
```

```
9
-6   -8   -2    6    4  -10   -7   10    9
6   -2   -6  -10    5    9    1   -4   10
7   -6    7   10   -3   10   -7    3    0
0    5    1    5    0   -6    1    9   -4
-5   -6   -9   -8   -1   -9    6    0    4
8   -8   -8   -6    7    0    4   -5   -9
3   -2    4   -7    6    7   -4    6    6
7    1    9    8    7   -3    7    3    8
2    4   -4   -8   -4   -5   -9   -9   10
```

Ожидаемый результат:

A[7x7]=

8	-8	4	4	1	-4	-1
-6	6	3	-10	-3	-8	9
-8	-3	8	-9	1	1	3
-4	6	8	2	-9	-2	-9
9	-9	-2	10	10	5	3
-1	-10	10	-9	-10	-6	6
9	-6	-7	-3	2	4	-5

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 7962624

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=0
oddRowSums[1]=9
oddRowSums[2]=-7
oddRowSums[3]=-18
oddRowSums[4]=8
oddRowSums[5]=-10
oddRowSums[6]=-6
```

$\max(\text{oddRowSums}) = \text{oddRowSums}[1] = 9$

B[9x9]=

-6	-8	-2	6	4	-10	-7	10	9
6	-2	-6	-10	5	9	1	-4	10
7	-6	7	10	-3	10	-7	3	0
0	5	1	5	0	-6	1	9	-4
-5	-6	-9	-8	-1	-9	6	0	4
8	-8	-8	-6	7	0	4	-5	-9
3	-2	4	-7	6	7	-4	6	6
7	1	9	8	7	-3	7	3	8
2	4	-4	-8	-4	-5	-9	-9	10

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 2144138240

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=2
oddRowSums[1]=15
oddRowSums[2]=7
oddRowSums[3]=21
oddRowSums[4]=-24
oddRowSums[5]=-7
oddRowSums[6]=3
oddRowSums[7]=31
oddRowSums[8]=-23
```

$\max(\text{oddRowSums}) = \text{oddRowSums}[7] = 31$

Результат:

A[7x7]=

8	-8	4	4	1	-4	-1
-6	6	3	-10	-3	-8	9
-8	-3	8	-9	1	1	3
-4	6	8	2	-9	-2	-9
9	-9	-2	10	10	5	3
-1	-10	10	-9	-10	-6	6
9	-6	-7	-3	2	4	-5

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 7962624

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=0
oddRowSums[1]=9
oddRowSums[2]=-7
oddRowSums[3]=-18
oddRowSums[4]=8
oddRowSums[5]=-10
oddRowSums[6]=-6
```

```
max(oddRowSums)=oddRowSums[1]=9
```

B[9x9]=

-6	-8	-2	6	4	-10	-7	10	9
6	-2	-6	-10	5	9	1	-4	10
7	-6	7	10	-3	10	-7	3	0
0	5	1	5	0	-6	1	9	-4
-5	-6	-9	-8	-1	-9	6	0	4
8	-8	-8	-6	7	0	4	-5	-9
3	-2	4	-7	6	7	-4	6	6
7	1	9	8	7	-3	7	3	8
2	4	-4	-8	-4	-5	-9	-9	10

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 2144138240

Задание 2.

Суммы по строкам нечётных элементов матрицы равны:

```
oddRowSums[0]=2
oddRowSums[1]=15
oddRowSums[2]=7
oddRowSums[3]=21
oddRowSums[4]=-24
oddRowSums[5]=-7
oddRowSums[6]=3
oddRowSums[7]=31
oddRowSums[8]=-23
```

```
max(oddRowSums)=oddRowSums[7]=31
```

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

Вывод

Разработка программы завершена на том основании, что:
полученные результаты совпали с ожидаемыми
считаем набор тестов полным